

布尔环线性因子增强的资源感知神经蒙特卡洛树搜索量子 oracle 综合

中文短论文稿 v3

2026 年 7 月 7 日

摘要

量子布尔函数 oracle 综合需要把经典谓词嵌入可逆量子线路，其逻辑层 T-count、CNOT、深度和辅助比特数会影响后续容错资源估计。直接从代数正规形 (algebraic normal form, ANF) 逐项生成线路语义清晰，但在高次数项和随机项集合中会重复计算大量 AND 结构。本文提出一条独立于 SSHR parallelootope 空间的路线：把 oracle 综合建模为 ANF/固定极性 Reed-Muller (fixed-polarity Reed-Muller, FPRM) 项集合上的资源约束搜索问题，并用神经动作先验、蒙特卡洛树搜索、baseline-preserving guard、Pareto portfolio 和布尔环线性因子共同生成低资源逻辑线路。当前证据表明，在 $n \leq 6$ 的 177 个传统函数上，Pareto-Resource-NMCTS 相对 ESOP cube beam 获得 174/0/3 的 score 胜/负/平，平均 score 降低 36.09%；相对 weighted ESOP-MILP 获得 167/3/7，平均 score 降低 29.84%。在 $n = 16$ 的 24 个随机 ANF 函数上，布尔环线性因子分支相对旧 recursive linear-pair guard 获得 17/3/4，平均 score 降低 0.39%；将该分支纳入 Resource-NMCTS 后，相对上一轮 pairwise-wide Resource-NMCTS 获得 14/0/10，平均 score 降低 0.34%。在 $n = 20$ 的 6 个边界函数上，传统 root beam 和 fast linear-pair 均全部超时，而 recursive Boolean-ring screen 使 Resource-NMCTS 相对旧 Resource-NMCTS 获得 5/0/1，平均 score 降低 4.52%；相对 AND-direct ANF 获得 5/0/1，平均 score 降低 9.04%。这些结果表明，本文方法可以作为一种逻辑层资源优化路线；同时，高维 learned prior 的质量增益仍小，外部 reversible-toolchain 对比仍需补齐，因此本文不主张硬件映射最优、CNOT-only 全面支配或全局最优。

关键词：量子 oracle 综合；布尔函数综合；神经蒙特卡洛树搜索；布尔环；ANF；FPRM；T-count

1 引言

给定布尔函数 $f : \{0, 1\}^n \rightarrow \{0, 1\}$ ，量子 oracle 通常实现为

$$|x\rangle|y\rangle \mapsto |x\rangle|y \oplus f(x)\rangle. \quad (1)$$

这类结构出现在 Grover 搜索、量子密码分析和多个量子算法子程序中。由于容错量子计算中非 Clifford 资源昂贵，oracle 的 T-count、T-depth、CNOT、逻辑深度和辅助比特数不能被视为后端细节，而是决定算法资源估计可信度的核心因素。

ANF 提供了最直接的综合入口。若

$$f(x) = \bigoplus_{m \in \mathcal{M}} \prod_{i \in m} x_i, \quad (2)$$

则每个单项式都可以被翻译为可逆乘法结构，并异或到输出位。然而，逐项翻译忽略了不同单项式之间的公共因子、线性奇偶结构和极性重写机会。对于随机高维 ANF，这种重复计算会迅速推高 T-count。

已有 XAG、ROS、BDD 和 SSHR 等方法从不同中间表示处理 oracle synthesis 问题 [1, 3, 4, 6]。本文的目标不是从 SSHR 出发，也不是把 SSHR 改写成 AI 版本，而是直接在 ANF/FPRM 项集合上搜索可复用因子。SSHR 在本文中只作为 CNOT-oriented small-scale baseline；本文方法内部不使用 parallelotope cover 候选空间。

本文的一句话论证是：在逻辑层量子布尔函数 oracle 综合中，ANF/FPRM 项集合上的资源感知神经树搜索，配合布尔环线性因子和保守 portfolio 选择，可以稳定降低 T-count 与加权逻辑资源；该优势的边界是更高 CNOT/辅助比特 trade-off、高维神经先验仍弱，以及尚未覆盖硬件映射。

2 相关工作与定位

2.1 布尔表示驱动的 oracle 综合

Xor-And-Inverter Graphs(XAG)把 XOR 与 AND 结构分离,使 AND 节点数量与非 Clifford 代价直接相关。Meuli 等人提出将 XAG 用于 quantum compilation[1], 并从 multiplicative complexity 角度解释低 T-count oracle 的来源 [2]。ROS 将 oracle synthesis 建模为 resource-constrained LUT mapping[3]。这些工作说明，中间表示的选择会决定可暴露的共享结构。本文与它们的共同点是重视符号布尔结构；差异在于本文不以 XAG 或 LUT 为唯一表示，而是在 ANF/FPRM 项集合上直接搜索因子和线性结构。

BDD-based reversible synthesis 使用决策图处理较大布尔函数 [4]。后端感知 oracle synthesis 进一步把 XAG 优化与后端质量指标连接起来 [5]。本文目前只处理逻辑层资源，不覆盖 routing、连通性、容错调度或真实噪声模型。

2.2 AI 搜索与线路综合

强化学习和树搜索已用于经典布尔电路、Clifford+T unitary、ZX-calculus rewrite 和量子线路 ordering 等任务 [7, 8, 9, 10]。这些工作支持“学习式搜索适合组合综合空间”这一大方向，但它们的动作空间与本文不同。本文动作不是一般 unitary gate、ZX rewrite rule 或 Pauli rotation ordering，而是 Boolean oracle 中 ANF/FPRM 项集合的因子选择、极性重写和 compute/uncompute 复用。

3 问题定义与资源模型

本文输入为布尔函数的真值表或 ANF 表示，输出为实现式 (1) 的逻辑层可逆线路。每条候选线路必须通过 truth-table 级语义验证。候选选择使用统一逻辑资源分数

$$\text{score} = T + 0.04 \text{ CNOT} + 0.015 \text{ depth} + 0.01 \text{ gates} + 2 \text{ ancilla}. \quad (3)$$

式 (3) 是可复现的逻辑层比较准则，不代表具体硬件平台的真实物理成本。报告结果时同时列出 T-count、CNOT、depth 和 peak ancilla，以避免单一 score 掩盖资源 trade-off。

表 1: 本文主张、证据和边界。

主张	证据	边界
低 T-count 与低 score	传统函数、ESOP/ABC/BDD/SSHR baseline、高维随机 ANF 压力测试	不等价于硬件映射后成本最小
方法独立于 SSHR	搜索空间为 ANF/FPRM 因子分解, 不使用 parallelotope cover	SSHR 仍是必要 CNOT baseline
搜索模块有独立贡献	仿射搜索、neural refine、guard、Pareto archive 和 high-dimensional linear-pair 消融	高维 learned prior 仍是小幅正向
布尔环线性因子改善高维结果	$n = 16$ matched comparison 和 $n = 20$ 边界测试	$n = 18$ ANF-only screen 质量较差, 不能泛化成深搜索结论

表 1 给出本文主张和边界。该表也是投稿时需要保持的论证纪律: 本文可以主张低 T-count 和低加权资源优势, 但不能主张所有资源维度同时最优。

4 方法

4.1 ANF/FPRM 项集合上的搜索状态

Resource-NMCTS 的搜索状态是当前尚未综合的项集合。动作是选择一个候选因子, 把局部项集合改写为一个可 compute/uncompute 的中间结构, 并递归处理剩余项。候选因子包括公共单项式因子、固定极性重写后的公共项、线性奇偶因子和布尔环线性因子。与直接从真值表训练黑盒模型相比, 这种状态定义保留了 Boolean algebra 结构, 也使每个动作可以被翻译为可验证的线路片段。

FPRM 极性搜索用于改变项集合形状。固定某些变量的极性后, 原始 ANF 中不明显的公共因子可能变为显式公共结构。低维传统函数上的大幅收益主要来自仿射坐标搜索、FPRM 极性和 portfolio 的组合, 而非单一 MCTS rollout。

4.2 布尔环线性因子

旧 linear-pair 分支主要搜索形如

$$(x_i \oplus x_j \oplus \dots) \cdot h(x) \quad (4)$$

的局部结构, 并要求线性因子变量与 quotient $h(x)$ 变量不相交。这个约束便于实现, 但会漏掉 square-free Boolean ring 中合法的重叠变量结构。本文新增的布尔环线性因子允许二者共享变量, 并在展开时使用 $x_i^2 = x_i$ 与 GF(2) 偶数项消去。例如

$$(x_i \oplus x_j)x_im = x_im \oplus x_ix_jm. \quad (5)$$

线路层面仍可先用 CNOT 计算线性奇偶量, 再用该辅助量控制 quotient 子线路。因此, 这一扩展不是单纯增加搜索宽度, 而是把 Boolean algebra 中原本被实现约束排除的候选重新纳入资源搜索。对于 $n = 20$ 这类 FPRM root beam 出现长尾的切片, 本文进一步使用 recursive Boolean-ring

screen: 先以 direct plan 为 baseline 搜索一层 ANF-only Boolean-ring 因子, 再在 quotient/rest 子问题上重复一次同类筛选, 从而避免极性枚举长尾, 同时保留 baseline-preserving 的候选选择。

4.3 神经动作先验、MCTS 与 guard

神经动作先验使用候选覆盖率、项数变化、平均次数、因子大小、即时资源收益和子问题规模等特征为动作排序。MCTS 在固定预算内探索候选组合, 并用 rollout 估计后续资源代价。低维 rerun 中 learned prior 给出无 score-loss 的小幅收益; 但高维中单独神经排序可能退化。因此本文采用 baseline-preserving guard: deterministic guard 与 neural guard 并行产生候选, 最终保留 score 更低者。

portfolio 层进一步合并 direct ANF、AND-direct ANF、FPRM root beam、linear-pair guard、Boolean-ring branch、Resource-NMCTS 与 Pareto-Resource-NMCTS 等候选。该设计的目标不是让每个模块都单独胜出, 而是在同一函数上避免明显差候选进入最终报告。

5 实验设计

实验包含四类证据。第一类为 $n \leq 6$ 的 177 个传统函数, 用于和 direct ANF、AND-direct ANF、ESOP cube beam、ESOP-MILP、SSHR-H、ABC 和 BDD baseline 比较。第二类为高维随机 ANF 压力测试, 包括 $n = 14$ 、 $n = 15$ 、 $n = 16$ 、 $n = 18$ 和 $n = 20$ 切片。第三类为搜索贡献分解, 用 matched function-level comparison 分离仿射搜索、神经 refine、guard、portfolio 和 Pareto archive 的作用。第四类为权重鲁棒性检查, 用 T-only、T-heavy、CNOT-depth 和 ancilla-tight 配置重算已验证结果。

所有内部结果均来自已经生成的 CSV、manifest 和分析脚本。高维切片中出现 timeout 的方法保留为错误行, 不参与 usable matched comparison; 这使 $n = 20$ 的结论更谨慎: 传统 root beam 和 fast linear-pair 无法在 300 s task timeout 内完成, 而 Boolean-ring screen 能给出可验证候选。

6 结果

6.1 传统函数上的主结果

表 2 给出 $n \leq 6$ 传统函数上的平均资源。相对 direct ANF, Pareto-Resource-NMCTS 平均 T-count 降低 72.25%, 平均 score 降低 67.80%。相对 ESOP cube beam, Pareto-Resource-NMCTS 获得 174/0/3 的 score 胜/负/平, 平均 score 降低 36.09%。相对 weighted ESOP-MILP, 获得 167/3/7, 平均 score 降低 29.84%。

SSHR-H 的平均 CNOT 最低, 说明本文优势不是 CNOT-only, 而是低 T-count 与低 weighted score。若评价准则更重视 CNOT-depth profile, 本文相对 SSHR-H 的优势会收窄, 但在当前权重鲁棒性分析中仍保持正向。

表 2: $n \leq 6$ 传统函数切片上的平均逻辑资源。

方法	平均 T	平均 CNOT	平均 depth	平均 ancilla	平均 score
Direct ANF	184.1	134.2	134.6	1.33	194.3
AND-direct ANF	101.0	166.5	166.9	2.18	115.2
Affine-NMCTS	45.9	94.4	98.9	1.88	55.4
Resource-NMCTS	43.9	89.9	94.5	1.92	53.2
Pareto-Resource-NMCTS	40.4	83.0	88.0	2.03	49.6
ESOP-MILP	83.6	133.5	159.6	2.32	96.7
SSHR-H	81.0	67.6	88.7	1.36	88.2

表 3: 搜索贡献分解。负的平均变化表示目标方法更优。

对比	score 胜/负/平	平均 Δ score
Affine greedy vs fixed-coordinate MCTS	165/88/68	-12.12%
Neural refine over affine greedy	65/0/257	-1.08%
Guarded Affine-NMCTS over no-guard	88/0/234	-1.74%
Resource-NMCTS over Affine-NMCTS	44/0/133	-1.91%
Pareto archive over Resource-NMCTS	68/0/109	-3.26%
Pareto-Resource-NMCTS over no-MCTS portfolio	106/0/71	-4.69%

6.2 搜索贡献分解

表 3 显示, 资源下降不是单个技巧造成的。仿射搜索提供最大前置收益; neural refine 和 guarded MCTS 在小规模切片中提供无 score-loss 小增益; Pareto archive 相对单一 Resource-NMCTS 进一步降低 score; 高维线性因子 guard 是 $n \geq 14$ 主要规模机制。

6.3 布尔环线性因子的高维收益

表 4 是本文当前最值得继续发展的新增证据。在 $n = 16$ 的 24 个 random ANF 函数上, Boolean-linear-deep 相对旧 recursive linear-pair guard 获得 17/3/4, 平均 score 降低 0.39%; 相对上一轮 pairwise-wide Resource-NMCTS 获得 14/6/4, 平均 score 降低 0.22%, 且平均运行时间降低 72.31%。将该分支纳入 Resource-NMCTS portfolio 后, Boolean-guard Resource-NMCTS 相对 pairwise-wide Resource-NMCTS 获得 14/0/10, 平均 score 降低 0.34%; 相对旧 deterministic recursive guard 获得 18/0/6, 平均 score 降低 0.52%。

这些数字的绝对幅度不大, 但可信度高于单纯神经排序诊断: 它同时改善 T-count、CNOT、depth 和 score, 并且在 portfolio 后没有 score loss。更重要的是, 它指出了后续应优先扩展的动作空间, 即让策略网络选择 Boolean-ring factor、递归深度和 FPRM polarity, 而不是只重排旧候选。

6.4 $n = 20$ 边界测试

$n = 20$ 切片包含 6 个 random ANF 函数、8 个方法和 48 行结果, 其中 12 行为 timeout。FPRM root beam 和 fast linear-pair 在所有 6 个函数上均触发 300 s task timeout。单层 ANF Boolean-ring

表 4: $n = 16$ 中布尔环线性因子的 matched comparison。

目标方法与 baseline	函数数	score 胜/负/平	平均 Δ score
Boolean-linear-deep vs old recursive guard	24	17/3/4	-0.39%
Boolean-linear-deep vs pairwise-wide Resource-NMCTS	24	14/6/4	-0.22%
Boolean-guard Resource-NMCTS vs pairwise-wide Resource-NMCTS	24	14/0/10	-0.34%
Boolean-guard Resource-NMCTS vs old recursive guard	24	18/0/6	-0.52%

表 5: $n = 20$ 边界测试。

比较	函数数	score 胜/负/平	平均 Δ score
Recursive-screen Resource-NMCTS vs old Resource-NMCTS	6	5/0/1	-4.52%
Recursive-screen Resource-NMCTS vs direct ANF	6	5/1/0	-37.10%
Recursive-screen Resource-NMCTS vs AND-direct ANF	6	5/0/1	-9.04%
Recursive Boolean screen vs single screen	6	5/0/1	-4.52%
FPRM root beam / fast linear-pair	6	全部 timeout	300 s task timeout

screen 已经使 Resource-NMCTS 相对 AND-direct ANF 获得 5/0/1, 平均 score 降低 4.89%。进一步加入 recursive Boolean-ring screen 后, 集成 Resource-NMCTS 相对旧 Resource-NMCTS 获得 5/0/1, 平均 score 降低 4.52%; 相对 AND-direct ANF 获得 5/0/1, 平均 score 降低 9.04%。表 5 给出核心比较。

该结果更适合解释为边界可扩展性证据, 而不是强神经搜索证据。 $n = 20$ 的收益主要来自递归布尔环 screen, 说明某些高维 ANF 可以通过极便宜的 Boolean algebra screen 获得改进; 但该结果仍不能说明 learned prior 已经能在 $n = 20$ 上做深搜索。

6.5 负面结果与不足

负面结果同样重要。 $n = 18$ 的 ANF-only Boolean-linear screen 虽然平均只需 0.843 s, 但相对旧 Resource-NMCTS 为 0/12/0, 平均 score 反而上升 42.45%。recursive screen 相对单层 screen 获得 11/0/1, 平均 score 降低 3.99%, 但相对旧 Resource-NMCTS 仍为 0/11/1, 平均 score 上升 36.94%。这说明 screen 版本可以作为快速边界候选, 不能替代 recursive Resource-NMCTS。

高维 learned prior 仍未达到理想强度。pairwise-wide root-action ranker 在 $n = 16$ full synthesis 上相对 recursive guard 获得 10/0/14, 平均 score 降低 0.18%, 属于可信小幅正向, 但还不是“明显提升”。本轮 Boolean-linear root-teacher 模型接入 baseline-preserving guard 后, 相对 deterministic Boolean-linear-deep 为 4/0/20, 平均 score 降低 0.12%, 但平均运行时间增加 94.49%。因此, 下一阶段的 AI 贡献必须从动作重排转向结构选择: 让模型预测何时使用 Boolean-ring factor、何时进入 FPRM 极性搜索、何时停止递归, 以及如何在 CNOT/depth/ancilla trade-off 中选择候选。

7 讨论

当前结果将本文定位为一篇逻辑层方法论文，核心不是“比 SSHR 强”，而是“不同搜索空间下的资源感知 Boolean oracle synthesis”。SSHR 的存在帮助暴露 CNOT trade-off; ESOP、ABC、BDD 和 XAG 文献帮助定位本文与传统布尔表示方法的关系；AI 搜索文献帮助说明 MCTS/learned prior 是合理的组合优化工具。本文真正需要强调的新点是：ANF/FPRM 项集合本身可以作为神经树搜索状态，布尔环线性因子能补上旧候选空间中的结构盲区。

进一步完善该路线仍需要三项内容。第一，补一个 reproduced ROS 或 mockturtle/RevKit 风格 reversible-toolchain baseline，以增强 external comparison 与 oracle synthesis 场景的贴合度。第二，补 2 到 3 个代表性线路案例，展示 direct ANF、root beam、recursive guard 和 Boolean-ring Resource-NMCTS 如何分解同一函数。第三，训练结构级策略网络，证明 AI 模块不仅能在旧候选上做小幅排序，还能预测高收益 Boolean-ring/FPRM 分支。

8 结论

本文表明，Resource-NMCTS 在 ANF/FPRM 项集合上进行资源感知搜索，配合 Pareto portfolio、baseline-preserving guard 和布尔环线性因子，可以在传统函数和高维随机 ANF 上降低逻辑层 T-count 与加权 score。布尔环线性因子为 $n = 16$ 提供了结构性 full-synthesis 改善；recursive Boolean-ring screen 则在 $n = 20$ 的 root-beam/fast-pair timeout 边界上继续降低旧 Resource-NMCTS 的 score。不过，高维 learned prior 的贡献仍偏小，外部 reversible-toolchain 对比和线路案例仍需补齐。后续工作应围绕结构级神经策略、可复现实验矩阵和英文正式稿展开。

参考文献

- [1] G. Meuli, M. Soeken, and G. De Micheli. Xor-And-Inverter Graphs for quantum compilation. *npj Quantum Information*, 8:7, 2022.
- [2] G. Meuli, M. Soeken, E. Campbell, M. Roetteler, and G. De Micheli. The role of multiplicative complexity in compiling low T-count oracle circuits. In *Proceedings of ICCAD*, 2019.
- [3] G. Meuli, M. Soeken, M. Roetteler, and G. De Micheli. ROS: Resource-constrained oracle synthesis for quantum computers. *Electronic Proceedings in Theoretical Computer Science*, 318:119–130, 2020.
- [4] R. Wille and R. Drechsler. BDD-based synthesis of reversible logic for large functions. In *Proceedings of DAC*, pp. 270–275, 2009.
- [5] M. Yu, A. Tempia Calvino, M. Soeken, and G. De Micheli. Back-end-aware fault-tolerant quantum oracle synthesis. In *Proceedings of ASP-DAC*, pp. 930–937, 2025.
- [6] Q. Zheng, Y. Xu, J. Zhang, Z. Su, and S. Zheng. CNOT oriented synthesis for small-scale Boolean functions using spatial structures of parallelotopes. In *Proceedings of ICCAD*, 2025.

- [7] D. Tsaras, A. Grosnit, L. Chen, Z. Xie, H. Bou-Ammar, and M. Yuan. ShortCircuit: AlphaZero-driven synthesis of Boolean circuits. *arXiv:2408.09858*, 2024.
- [8] S. Rietsch, A. Y. Dubey, C. Ufrecht, M. Periyasamy, A. Plinge, C. Mutschler, and D. D. Scherer. Unitary synthesis of Clifford+T circuits with reinforcement learning. In *IEEE International Conference on Quantum Computing and Engineering*, pp. 824–835, 2024.
- [9] J. Riu, J. Nogu  , G. Vilaplana, A. Garcia-Saez, and M. P. Estarellas. Reinforcement learning based quantum circuit optimization via ZX-calculus. *Quantum*, 9:1758, 2025.
- [10] M. Machiya, M. Menickelly, P. Hovland, and J. Liu. MonteQ: A Monte Carlo tree search based quantum circuit synthesis framework. *arXiv:2604.19029*, 2026.